



Amazon Sales Dataset

Data Science Project Pipeline

End-to-End Guide: **Cleaning** → **EDA** → **Modeling** → **Streamlit Dashboard**

100K	Dataset Rows
17	Features
6	Pipeline Phases
1	Live Dashboard

Prepared for:	Data Science Practitioners
Dataset:	Amazon Sales (Orders, Customers, Products, Revenue)
Tech Stack:	Python • Pandas • Scikit-learn • Streamlit • Plotly
Date:	May 01, 2026

Table of Contents

1.	Project Overview & Goals	3
2.	Project Structure & File Organization	4
3.	Report Structure	5
4.	Phase 1 – Data Cleaning & Preprocessing	6
5.	Phase 2 – Exploratory Data Analysis	8
6.	Phase 3 – Feature Engineering	10
7.	Phase 4 – Machine Learning Modeling	11
8.	Phase 5 – Streamlit Dashboard	13
9.	Phase 6 – Reporting & Decision Making	15
10.	Tools, Libraries & Environment Setup	16
11.	Best Practices & Tips	17

1. Project Overview & Goals

This document serves as the authoritative reference for building a complete Data Science project on the Amazon Sales Dataset (100,000 rows, 17 columns). It walks you through every stage of the analytical pipeline — from raw data ingestion to a production-ready Streamlit dashboard that empowers business stakeholders to make data-driven decisions.

Business Questions We Will Answer

- Which product categories and brands drive the most revenue?
- What is the impact of discounts and shipping costs on profitability?
- Which cities / states / countries are the top-performing markets?
- What payment methods are most popular, and do they correlate with order value?
- Can we predict TotalAmount (revenue) from order and product features?
- Are there customer segments that behave differently (RFM Analysis)?
- What is the monthly/seasonal sales trend and can we forecast it?

Success Criteria

Criterion	Target
Data Quality	< 1% missing values after cleaning, 0 duplicates
EDA Coverage	All 17 columns explored with visualizations
Model Performance	$R^2 \geq 0.80$ for revenue prediction
Dashboard	Interactive Streamlit app with 5+ chart types
Reporting	Clear business recommendations with evidence

2. Project Structure & File Organization

A clean, reproducible folder structure is essential for collaboration, version control, and deployment. Follow the structure below from day one.

```
amazon_sales_ds/
├── data/
│   ├── raw/           # Original unmodified CSV
│   ├── processed/     # Cleaned & feature-engineered datasets
│   └── external/      # Optional: geo or macro data
├── notebooks/
│   ├── 01_data_cleaning.ipynb
│   ├── 02_eda.ipynb
│   ├── 03_feature_engineering.ipynb
│   ├── 04_modeling.ipynb
│   └── 05_insights.ipynb
├── src/
│   ├── __init__.py
│   ├── cleaning.py    # Reusable cleaning functions
│   ├── features.py    # Feature engineering functions
│   ├── modeling.py    # Model train / eval helpers
│   └── viz.py         # Plotly / Matplotlib helpers
├── streamlit_app/
│   ├── app.py         # Main Streamlit entry point
│   └── pages/
│       ├── 1_Overview.py
│       ├── 2_Sales_Analysis.py
│       ├── 3_Customer_Insights.py
│       ├── 4_Product_Performance.py
│       └── 5_Predictions.py
├── assets/           # Logos, CSS
├── models/           # Saved .pkl / .joblib model files
├── reports/
│   ├── figures/      # Saved charts (PNG/SVG)
│   └── final_report.pdf
├── requirements.txt
├── README.md
└── .gitignore
```

Tip: Initialize a Git repository immediately (**git init**) and commit each phase separately. Use **DVC** (Data Version Control) to track large data files without storing them in Git.

3. Report Structure

Your final project report should follow a standard data science narrative. Below is the recommended structure for a professional, publication-ready report.

1. Executive Summary

1–2 pages. High-level findings, key metrics, and top 3 recommendations. Written last but placed first. Non-technical audience.

2. Problem Statement

Define the business context, analytical objectives, and success metrics. State the dataset source, size, and time period.

3. Data Description

Schema overview (all 17 columns with type, range, and meaning). Dataset statistics: row count, missing values, duplicates.

4. Data Cleaning

Document every transformation: null handling, type casting, outlier treatment, deduplication. Before/after comparison tables.

5. Exploratory Data Analysis

Key findings per variable. Distribution plots, correlation matrix, top-N rankings. Minimum 15 visualizations.

6. Feature Engineering

List all derived features with formulas. Explain why each feature adds predictive or analytical value.

7. Modeling

Model selection rationale, hyperparameter tuning, evaluation metrics (RMSE, R^2 , MAE). Feature importance chart.

8. Dashboard Overview

Screenshots of each Streamlit page. Usage guide for non-technical users.

9. Business Recommendations

3–5 actionable recommendations backed by data. Each with estimated impact and implementation effort.

10. Limitations & Next Steps

Data gaps, model assumptions, proposed future work (e.g., real-time data, A/B testing, NLP on product names).

11. Appendices

Full code listings, additional charts, data dictionary, and model training logs.

Phase 1 — Data Cleaning & Preprocessing

4. Phase 1 – Data Cleaning & Preprocessing

Data cleaning is the most time-consuming phase (~30–40% of total effort) but the most impactful. On a 100k-row e-commerce dataset, common issues include mixed date formats, negative prices, duplicate orders, and inconsistent country/city spellings.

4.1 Initial Data Audit

- `df.info()` → column types and non-null counts
- `df.describe()` → numeric distributions (min, max, mean, std)
- `df.isnull().sum()` → missing value count per column
- `df.duplicated().sum()` → exact duplicate rows
- `df.nunique()` → cardinality of categorical columns
- `df['OrderDate'].dtype` → confirm datetime vs object

4.2 Missing Value Treatment Strategy

Column Type	Strategy	Rationale
Numerical (price)	Median imputation	Robust to outliers
Categorical	Mode imputation / 'Unknown'	Avoids information loss
Dates	Drop row if critical	Imputing dates is unreliable
ID columns	Drop row	Cannot infer IDs
Discount / Tax	Fill 0 if NaN	Absence = no discount/tax

4.3 Key Cleaning Steps (Code Reference)

```
import pandas as pd
import numpy as np

df = pd.read_csv('data/raw/amazon_sales.csv')

# 1. Parse dates
df['OrderDate'] = pd.to_datetime(df['OrderDate'], errors='coerce')

# 2. Drop full duplicates
df.drop_duplicates(inplace=True)

# 3. Remove impossible values
df = df[df['UnitPrice'] > 0]
df = df[df['Quantity'] > 0]
df = df[(df['Discount'] >= 0) & (df['Discount'] <= 1)]

# 4. Standardize text columns
for col in ['City', 'State', 'Country', 'Category', 'Brand']:
    df[col] = df[col].str.strip().str.title()

# 5. Validate TotalAmount
```

```
df['CalcTotal'] = (df['UnitPrice'] * df['Quantity'] *  
                  (1 - df['Discount'])) + df['Tax'] + df['ShippingCost'])  
df['AmountMismatch'] = abs(df['TotalAmount'] - df['CalcTotal']) > 0.01  
  
# 6. Handle outliers (IQR method on UnitPrice)  
Q1, Q3 = df['UnitPrice'].quantile([0.25, 0.75])  
IQR = Q3 - Q1  
df = df[df['UnitPrice'].between(Q1 - 3*IQR, Q3 + 3*IQR)]  
  
df.to_parquet('data/processed/cleaned.parquet', index=False)
```

Why Parquet? Save cleaned data as Parquet instead of CSV. Parquet preserves dtypes (including datetime), compresses ~5× smaller, and loads 10–20× faster — critical for a 100k-row dataset in Streamlit.

Phase 2 — Exploratory Data Analysis (EDA)

5. Phase 2 – Exploratory Data Analysis

EDA transforms cleaned data into understanding. Every column deserves at least one visualization. Structure your EDA notebook around the five analytical lenses below.

5.1 Univariate Analysis

- OrderStatus → countplot (bar chart of status distribution)
- TotalAmount → histogram + KDE (identify skewness)
- Quantity → box plot (spot outliers)
- Category / Brand → horizontal bar chart (top-20 by count)
- PaymentMethod → pie chart
- Country / State / City → choropleth map (Plotly Express)

5.2 Bivariate Analysis

- TotalAmount vs UnitPrice → scatter plot colored by Category
- Discount vs TotalAmount → scatter + regression line
- Category vs TotalAmount → grouped box plot
- PaymentMethod vs TotalAmount → violin plot
- OrderStatus vs TotalAmount → stacked bar chart
- Country vs AvgOrderValue → sorted horizontal bar chart

5.3 Time Series Analysis

- Monthly revenue trend → line chart (resample by month)
- Day-of-week order volume → bar chart
- Hour-of-day (if available) → heatmap
- Rolling 30-day average revenue → overlay on monthly line
- Year-over-Year growth % → waterfall chart

5.4 Correlation Analysis

- Pearson correlation heatmap of all numeric columns
- Scatter matrix (pairplot) for: UnitPrice, Quantity, Discount, Tax, ShippingCost, TotalAmount
- VIF (Variance Inflation Factor) to detect multicollinearity before modeling

5.5 Geo Analysis

- Plotly choropleth: TotalRevenue by Country
- Plotly scatter_geo: top-50 cities by order volume
- Heatmap: Category preference by Country (pivot table)

EDA Checklist: Before moving to Phase 3, confirm: (1) all columns explored, (2) ≥ 3 non-obvious insights documented, (3) correlation heatmap reviewed, (4) at least one geo visualization created, (5) time series plotted.

Phase 3 — Feature Engineering

6. Phase 3 – Feature Engineering

Feature engineering is the bridge between raw data and high-performing models. Good features encode domain knowledge. Below are all recommended derived features for the Amazon Sales dataset.

Feature Name	Formula / Logic	Type
OrderYear	df['OrderDate'].dt.year	Temporal
OrderMonth	df['OrderDate'].dt.month	Temporal
OrderDayOfWeek	df['OrderDate'].dt.dayofweek	Temporal
IsWeekend	OrderDayOfWeek >= 5	Binary
Quarter	df['OrderDate'].dt.quarter	Temporal
Revenue	UnitPrice * Quantity * (1 - Discount)	Financial
DiscountAmount	UnitPrice * Quantity * Discount	Financial
ProfitProxy	Revenue - ShippingCost - Tax	Financial
PricePerUnit	TotalAmount / Quantity	Derived
ShippingRatio	ShippingCost / TotalAmount	Ratio
IsDiscounted	Discount > 0	Binary
HighValueOrder	TotalAmount > 75th percentile	Binary
CustomerOrderCount	groupby(CustomerID).transform('count')	Aggregated
CustomerAvgSpend	groupby(CustomerID)['TotalAmount'].mean()	Aggregated
CategoryAvgPrice	groupby(Category)['UnitPrice'].mean()	Aggregated
BrandPopularity	groupby(Brand).transform('count')	Aggregated

RFM Segmentation (Customer Intelligence)

RFM (Recency, Frequency, Monetary) analysis segments customers by purchasing behaviour. Assign each customer a score of 1–5 on each dimension, then combine into a segment label.

```
snapshot = df['OrderDate'].max() + pd.Timedelta(days=1)
rfm = df.groupby('CustomerID').agg(
    Recency = ('OrderDate', lambda x: (snapshot - x.max()).days),
    Frequency = ('OrderID', 'count'),
    Monetary = ('TotalAmount', 'sum')
).reset_index()

# Score each dimension 1-5 (5 = best)
rfm['R_Score'] = pd.qcut(rfm['Recency'], 5, labels=[5,4,3,2,1]).astype(int)
rfm['F_Score'] = pd.qcut(rfm['Frequency'], 5, labels=[1,2,3,4,5]).astype(int)
rfm['M_Score'] = pd.qcut(rfm['Monetary'], 5, labels=[1,2,3,4,5]).astype(int)
rfm['RFM_Segment'] = rfm['R_Score'].astype(str) + rfm['F_Score'].astype(str) + rfm['M_Score'].a
stype(str)
```

Phase 4 — Machine Learning Modeling

7. Phase 4 – Machine Learning Modeling

This phase covers three modeling tasks: regression (predict TotalAmount), classification (predict OrderStatus), and clustering (customer segmentation). Use a reproducible pipeline with scikit-learn.

7.1 Modeling Tasks Summary

Task	Target Variable	Algorithm(s)	Metric
Revenue Prediction	TotalAmount	Linear Reg, Random Forest, XGBoost	RMSE, R ²
Status Prediction	OrderStatus	Logistic Reg, Gradient Boosting	F1, AUC-ROC
Customer Segments	No target (unsup)	K-Means, DBSCAN	Silhouette Score
Sales Forecasting	Monthly Revenue	Prophet, ARIMA	MAPE, MAE

7.2 Scikit-learn Pipeline Template

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import mean_squared_error, r2_score
import joblib

NUM_COLS = ['UnitPrice','Quantity','Discount','Tax','ShippingCost',
            'OrderMonth','OrderDayOfWeek','CustomerAvgSpend']
CAT_COLS = ['Category','Brand','PaymentMethod','Country']

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), NUM_COLS),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), CAT_COLS)
])

pipeline = Pipeline([
    ('prep', preprocessor),
    ('model', RandomForestRegressor(n_estimators=200, random_state=42))
])

X = df[NUM_COLS + CAT_COLS]
y = df['TotalAmount']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
print(f'R²: {r2_score(y_test, y_pred):.4f}')
print(f'RMSE: {mean_squared_error(y_test, y_pred, squared=False):.2f}')

joblib.dump(pipeline, 'models/rf_revenue_pipeline.joblib')
```

7.3 Hyperparameter Tuning

Use `RandomizedSearchCV` (faster than `GridSearchCV` on large datasets) with 5-fold cross-validation. For XGBoost, additionally use early stopping on a validation split to prevent overfitting.

```
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    'model__n_estimators': [100, 200, 300, 500],
    'model__max_depth': [None, 10, 20, 30],
    'model__min_samples_split': [2, 5, 10],
    'model__max_features': ['sqrt', 'log2', 0.5]
}

search = RandomizedSearchCV(pipeline, param_dist, n_iter=30,
                           cv=5, scoring='r2', n_jobs=-1, random_state=42)
search.fit(X_train, y_train)
print('Best params:', search.best_params_)
print('Best CV R²:', search.best_score_)
```

Phase 5 — Streamlit Dashboard

8. Phase 5 – Streamlit Dashboard

The Streamlit dashboard is the primary deliverable for business stakeholders. It transforms your analysis into an interactive decision-support tool. Structure it as a multi-page app using Streamlit's native pages feature.

8.1 Page-by-Page Specification

Page 1: ■ Executive Overview

- KPI cards: Total Revenue, Total Orders, Avg Order Value, Return Rate
- Revenue trend line chart with month filter
- Top-5 categories (donut chart) + Top-5 countries (bar chart)
- Date range selector (sidebar) affecting all charts on page

Page 2: ■ Sales Analysis

- Revenue breakdown by Category, Brand, PaymentMethod (tabs)
- Discount impact: scatter plot (Discount vs Revenue)
- Shipping cost analysis: box plot by Country
- Monthly heatmap (rows = months, cols = categories)

Page 3: ■ Customer Insights

- RFM segment distribution (scatter: Recency vs Frequency, size = Monetary)
- Top-20 customers by lifetime value (horizontal bar)
- Customer count by City / State (choropleth map)
- Average order value by payment method (grouped bar)

Page 4: ■ Product Performance

- Best-selling products (top-20 by revenue and by quantity)
- Category margin analysis (Revenue - ShippingCost - Tax)
- Brand comparison: avg unit price vs avg quantity ordered
- Product filter: search bar → individual product drilldown

Page 5: ■ Revenue Predictor

- Input form: Category, Brand, UnitPrice, Quantity, Discount, Country, PaymentMethod
- Predict button → calls loaded joblib model → displays predicted TotalAmount
- SHAP waterfall chart showing feature contributions to prediction
- Model performance summary (R^2 , RMSE, MAE on test set)

8.2 Main app.py Skeleton

```
import streamlit as st
import pandas as pd
import plotly.express as px
import joblib
```

```
st.set_page_config(page_title='Amazon Sales DS', layout='wide',
                    page_icon='■', initial_sidebar_state='expanded')

@st.cache_data
def load_data():
    return pd.read_parquet('data/processed/cleaned.parquet')

@st.cache_resource
def load_model():
    return joblib.load('models/rf_revenue_pipeline.joblib')

df = load_data()
model = load_model()

# Sidebar global filters
st.sidebar.header('Global Filters')
date_range = st.sidebar.date_input('Date Range',
    value=[df['OrderDate'].min(), df['OrderDate'].max()])
categories = st.sidebar.multiselect('Categories', df['Category'].unique(),
    default=df['Category'].unique())

mask = (
    (df['OrderDate'] >= pd.Timestamp(date_range[0])) &
    (df['OrderDate'] <= pd.Timestamp(date_range[1])) &
    (df['Category'].isin(categories))
)
filtered = df[mask]

# KPI Row
c1, c2, c3, c4 = st.columns(4)
c1.metric('Total Revenue', f"${filtered['TotalAmount'].sum():,.0f}")
c2.metric('Total Orders', f"{len(filtered):,}")
c3.metric('Avg Order Value', f"${filtered['TotalAmount'].mean():.2f}")
c4.metric('Countries', filtered['Country'].nunique())
```

Phase 6 — Reporting & Decision Making

9. Phase 6 – Reporting & Decision Making

Raw insights have no value until they change behaviour. This phase converts analytical findings into concrete business recommendations presented to decision-makers.

9.1 Decision Framework

Finding Type	Recommendation Format	Audience	Timeline
Quick Win (< 1 month)	Slack / Email Memo	Operations	Immediate
Tactical (1–3 months)	1-page Briefing	Department Head	Monthly Review
Strategic (3–12 mo)	Full Report + Dashboard	C-Suite	Quarterly
Product Change	A/B Test Proposal	Product Team	Sprint Planning

9.2 Sample Recommendations Template

- **Recommendation 1:** Increase promotion budget for top-3 categories by 15% → expected revenue lift of \$X based on elasticity analysis.
- **Recommendation 2:** Introduce free shipping threshold at \$50 → 12% of orders are just below this amount; converting them increases AOV.
- **Recommendation 3:** Target Champions segment (RFM = 555) with loyalty programme → they represent 20% of revenue from 5% of customers.
- **Recommendation 4:** Investigate high-discount orders with negative ProfitProxy → apply minimum margin guardrails in pricing engine.
- **Recommendation 5:** Expand into top-3 underperforming countries with strong demand but low seller presence.

10. Tools, Libraries & Environment Setup

10.1 requirements.txt

```
# Core
pandas==2.2.0
numpy==1.26.4
scipy==1.12.0

# Visualization
matplotlib==3.8.3
seaborn==0.13.2
plotly==5.20.0

# Machine Learning
scikit-learn==1.4.1
xgboost==2.0.3
lightgbm==4.3.0
shap==0.45.0

# Time Series
prophet==1.1.5
statsmodels==0.14.1

# Dashboard
streamlit==1.32.0

# Utilities
joblib==1.3.2
pyarrow==15.0.0
python-dotenv==1.0.1
```

10.2 Virtual Environment Setup

```
# Create environment
python -m venv .env
source .env/bin/activate      # Windows: .env\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Launch Streamlit
cd streamlit_app
streamlit run app.py

# Launch JupyterLab
jupyter lab notebooks/
```

11. Best Practices & Tips

Version Control

- Commit after each phase with a descriptive message (e.g., 'feat: add RFM segmentation')
- Use .gitignore to exclude data/, models/, and .venv/
- Tag milestones: git tag v1.0-cleaned, v2.0-eda, etc.

Reproducibility

- Set random_state=42 in all stochastic functions
- Pin library versions in requirements.txt
- Use a config.py for all file paths and hyperparameters

Performance

- Use @st.cache_data and @st.cache_resource in Streamlit aggressively
- Downsample large datasets for exploratory plots (sample 10k rows for scatter plots)
- Use Parquet with partition_by='OrderYear' for faster filtered reads

Documentation

- Write a README.md with: project description, setup instructions, and one-liner commands
- Document every non-obvious function with a docstring
- Export at least 5 high-resolution figures (300 DPI) for the final report

Presentation

- Lead with the insight, not the methodology — executives care about 'what' not 'how'
- Every chart must have: title, axis labels, source note, and a one-sentence takeaway
- Use consistent color schemes (orange for Amazon, blue for comparisons, red for alerts)

Pipeline at a Glance

#	Phase	Duration	Key Output
1	Data Cleaning	1–2 days	cleaned.parquet
2	EDA	2–3 days	15+ visualizations, insights doc
3	Feature Engineering	1 day	features.parquet, rfm.parquet
4	Modeling	2–3 days	rf_pipeline.joblib, metrics report

5	Streamlit Dashboard	2–3 days	Running app on localhost:8501
6	Reporting	1–2 days	final_report.pdf, slide deck
—	TOTAL	9–14 days	Full end-to-end DS project

You are now ready to begin. Open your terminal, activate your virtual environment, and start with **01_data_cleaning.ipynb**. Remember: good data science is iterative — expect to revisit earlier phases as you learn more about the data. Good luck! ■